Ответы на контрольные вопросы

Проект «КиноПлан» — сервис планирования киносъёмок (Django 5.2)

1. Структура проекта Django

Покажите конфигурационный пакет Django-проекта. Какие задачи решают settings.py и корневой urls.py в работе приложения?

Конфигурационный пакет проекта — каталог `kinoplan/`, созданный командой `django-admin startproject kinoplan .`:

```
film_scheduler/
├── manage.py
├── kinoplan/          ← конфигурационный пакет
│   ├── __init__.py    ← делает каталог Python-пакетом
│   ├── settings.py    ← настройки проекта
│   ├── urls.py        ← корневой URLconf
│   ├── wsgi.py        ← точка входа для WSGI-сервера
│   └── asgi.py        ← точка входа для ASGI-сервера
├── homepage/ films/ locations/ shootings/  ← приложения
├── models/  storage.py  data/   ← классы и JSON-хранилище
└── db.sqlite3
```

settings.py — единое место конфигурации всего проекта. В нём задаются:

- `BASE_DIR` — корень проекта, от него строятся остальные пути;
- `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS` — безопасность и режим работы;
- `INSTALLED_APPS` — какие приложения входят в проект (в т.ч. `homepage`, `films`, `locations`, `shootings`);
- `MIDDLEWARE` — цепочка промежуточных обработчиков запроса/ответа (CSRF, сессии, проверка хоста и т.д.);
- `ROOT_URLCONF = 'kinoplan.urls'` — с какого файла начинать разбор адреса;
- `TEMPLATES`, `DATABASES` (SQLite), `STATIC_URL`;
- локализация: `LANGUAGE_CODE = "ru-RU"`, `TIME_ZONE = "Europe/Moscow"`.

Корневой urls.py — точка входа маршрутизации. Он не содержит адресов конкретных страниц, а распределяет запросы по приложениям через `include()` и подключает админку и обработчик 404:

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("homepage.urls")),
    path("films/", include("films.urls")),
    path("locations/", include("locations.urls")),
    path("shootings/", include("shootings.urls")),
]

handler404 = "homepage.views.page_not_found"
```

2. Проверка и запуск проекта

Покажите manage.py. Какой командой проверить конфигурацию проекта, а какой — запустить сервер? Каково назначение этого файла?

```
def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'kinoplan.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError("Couldn't import Django. ...") from exc
    execute_from_command_line(sys.argv)
```

Назначение. `manage.py` — утилита командной строки для управления конкретным проектом. Она устанавливает переменную окружения `DJANGO_SETTINGS_MODULE = 'kinoplan.settings'` (т.е. сообщает Django, какие настройки использовать) и передаёт аргументы командной строки в `execute_from_command_line()`. Через неё выполняются все административные команды: `check`, `runserver`, `startapp`, `makemigrations`, `migrate`, `createsuperuser`, `shell` и др.

Действие	**Команда**
Проверить конфигурацию	`python manage.py check` (для боевых настроек — `python manage.py check --deploy`)
Запустить сервер разработки	`python manage.py runserver` → http://127.0.0.1:8000/

Результат проверки в проекте: `System check identified no issues (0 silenced).`

3. Регистрация приложений

Покажите регистрацию приложений в INSTALLED_APPS. Что может произойти, если приложение создано, но не зарегистрировано в настройках проекта?

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'homepage',
    'films',
    'locations',
    'shootings',
]
```

Первые шесть — встроенные приложения Django, последние четыре — собственные приложения КиноПлана.

Если приложение создано (`startapp`), но не добавлено в `INSTALLED_APPS`, Django его «не видит» как часть проекта:

- его модели не регистрируются: `makemigrations`/`migrate` их не обрабатывают, таблицы не создаются, а при импорте модели возникает `RuntimeError: Model class ... doesn't declare an explicit app_label and isn't in an application in INSTALLED_APPS`;
- не ищутся шаблоны в `<app>/templates/` (при `APP_DIRS = True`) и статика в `<app>/static/` → `TemplateDoesNotExist`, ненайденные CSS/картинки;
- не загружаются `admin.py` (модели не появятся в админке), `apps.py` (метод `ready()`, сигналы), команды management;
- тесты приложения могут не запускаться корректно.

Важно: в текущей версии КиноПлана приложения не используют модели БД и шаблоны (данные берутся из JSON через `storage.py`, HTML строится функцией `page()`). Поэтому, если убрать, например, `'films'` из `INSTALLED_APPS`, страницы `/films/` продолжат открываться — маршрут подключён через `include("films.urls")`, а это обычный импорт Python-модуля. Ошибка проявится позже, как только в приложении появятся модели, шаблоны или админка. Поэтому приложение регистрируется сразу после создания.

4. Структура приложений КиноПлана

Какие Django-приложения созданы и как распределены между ними страницы? Почему в рамках текущей реализации отдельно не создавалось приложение для пользователей?

Проект разделён на приложения по разделам предметной области — каждое отвечает за одну сущность:

Приложение	Адрес	View-функция	Страница
`homepage`	`/`	`index`	главная: описание сервиса, счётчики фильмов, локаций, активных съёмок
`films`	`/films/`	`films`	список фильмов с числом сцен
	`/films/<int:film_id>/`	`film_detail`	сцены фильма и его съёмки
`locations`	`/locations/`	`locations`	таблица локаций по убыванию вместимости
	`/locations/<int:location_id>/`	`location_detail`	карточка локации, проверка занятости на дату (`?date=ГГГГ-ММ-ДД`)

shootings	/shootings/	shootings	график съёмок по датам со статусами
	/shootings/<int:shooting_id>/	shooting_detail	карточка съёмки со ссылками на фильм и локацию

Кроме того, homepage содержит общие для всех приложений функции: page() (HTML-каркас с Bootstrap и навигацией), not_found_page() и page_not_found() (обработчик 404). Классы предметной области (Film, Scene, Location, Shooting) вынесены в пакет models/, работа с JSON — в storage.py; они общие для веб-версии и консольной main.py.

Почему нет приложения для пользователей:

- в текущей реализации нет регистрации, входа и личных кабинетов — все страницы публичные и только отображают данные; функций, привязанных к конкретному пользователю, пока нет;
- пользователи (режиссёры, ассистенты, продюсеры) не являются сущностью предметной области на данном этапе: данные о фильмах, локациях и съёмках не хранят автора;
- базовая работа с пользователями уже есть во встроенном приложении django.contrib.auth (модель User, группы, права, админка) — дублировать её нет смысла;
- пустое приложение «на будущее» только усложнило бы структуру. Отдельное приложение (например, accounts) появится, когда понадобятся авторизация, роли и разграничение прав на редактирование графика.

5. Настройки режима отладки

Что необходимо проверить или изменить в настройках перед отключением режима отладки? Как параметры DEBUG и ALLOWED_HOSTS влияют на работу проекта?

Текущие значения в kinoplan/settings.py:

```
DEBUG = True
ALLOWED_HOSTS = ["localhost", "127.0.0.1"]
```

DEBUG — режим отладки.

- True: при ошибке показывается подробная жёлтая страница с трассировкой, значениями переменных и настроек; при 404 — список всех маршрутов проекта; сервер разработки сам раздаёт статические файлы. Удобно при разработке, но раскрывает внутреннее устройство приложения.
- False: пользователь видит только короткую страницу ошибки; для 404 используется собственный обработчик handler404 = "homepage.views.page_not_found"; статика через runserver не раздаётся.

ALLOWED_HOSTS — список доменов/IP, по которым разрешено обращаться к сайту (сравнивается с заголовком Host запроса). Запрос с другим хостом получает ответ **400 Bad**

Request (`DisallowedHost`). Это защита от подмены заголовка Host. При `DEBUG = True` и пустом списке Django автоматически разрешает `localhost`, `127.0.0.1`, `[::1]`; при `DEBUG = False` пустой список недопустим — `runserver` не запустится с ошибкой `CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.`

Проверено на проекте: обращение тестового клиента с хостом `testserver`, которого нет в списке, вернуло код 400 с сообщением `Invalid HTTP_HOST header: 'testserver'. You may need to add 'testserver' to ALLOWED_HOSTS.`

Что проверить перед `DEBUG = False`:

1. `ALLOWED_HOSTS` не пуст и содержит все адреса, по которым открывается сайт (у нас `localhost`, `127.0.0.1`; на сервере — его домен/IP).
2. Страница 404 корректно работает: `handler404` указывает на существующую функцию `page_not_found(request, exception)` с правильной сигнатурой.
3. Статические файлы: задать `STATIC_ROOT` и выполнить `collectstatic` либо раздавать их веб-сервером (в КиноПлане Bootstrap подключается с CDN, поэтому оформление не пропадёт).
4. `SECRET_KEY` заменить на случайный и не хранить в коде (переменная окружения), т.к. текущий ключ с префиксом `django-insecure-`.
5. Запустить `python manage.py check --deploy` и устранить предупреждения (HTTPS, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE` и т.п.).
6. Убедиться, что ошибки логируются (`LOGGING`, `ADMINS`), так как трассировка больше не выводится на страницу.

6. Маршрутизация между файлами URLconf

Проследите разрешение URL /locations/999/ от корневого urls.py до view-функции приложения locations. Какие части адреса обрабатываются в каждом файле и для чего используется include()?

Шаг 0. Django берёт путь запроса без ведущего слэша — `locations/999/` — и по настройке `ROOT_URLCONF = 'kinoplan.urls'` открывает корневой URLconf.

Шаг 1. kinoplan/urls.py — шаблоны проверяются сверху вниз:

```
path("admin/", admin.site.urls),              # не совпало
path("", include("homepage.urls")),           # в homepage.urls только ""
path("films/", include("films.urls")),        # не совпало
path("locations/", include("locations.urls")), # совпал префикс "locations/"
```

Здесь обрабатывается только префикс `locations/`. Он «отрезается», а оставшаяся часть `999/` передаётся в подключённый URLconf приложения.

Шаг 2. locations/urls.py — разбирает остаток `999/`:

```
urlpatterns = [
    path("", views.locations, name="locations"),           # "" ≠ "999/"
    path("<int:location_id>/", views.location_detail,
         name="location_detail"),                          # совпало
]
```

Конвертер `<int:location_id>` проверяет, что сегмент состоит из цифр, преобразует строку `"999"` в число `999` и передаёт его в view как именованный аргумент.

Шаг 3. locations/views.py — вызывается `location_detail(request, location_id=999)`. Функция загружает локации из `data/locations.json`, ищет `find_location_by_id(locations_list, 999)`, не находит её и возвращает страницу «Локация не найдена» с кодом **404** и ссылкой «к списку локаций».

Проверка через `django.urls.resolve('/locations/999/')`:

```
func:   locations.views.location_detail
kwargs: {'location_id': 999}
route:  locations/<int:location_id>/
name:   location_detail
ответ:  404, «Локация не найдена»
```

Для сравнения: `/locations/abc/` не подходит под конвертер `int`, ни один шаблон не совпадает, и 404 возвращает уже сам маршрутизатор Django, view не вызывается вообще.

Зачем нужен include(): он подключает список маршрутов другого модуля под общим префиксом. Благодаря этому:

- корневой `urls.py` остаётся коротким и отвечает только за распределение по разделам, а каждое приложение само описывает свои внутренние адреса;
- приложения независимы: `locations/urls.py` ничего не знает о префиксе `locations/`, и его можно сменить в одном месте, не трогая приложение;
- проще развивать и переиспользовать приложения в других проектах.